

SYN Flood Attack Investigation Using tshark

OJE DOMINION MAYOWA

FWSD2511507

Basic Networking Skills for Digital Forensics

MR. Aminu Idris

11 September 2026

LAB OVERVIEW

This lab compared a normal HTTP handshake against a bounded SYN probe simulation to identify indicators of incomplete or suspicious TCP activity. I captured a legitimate curl-based session as a baseline, then used a Scapy script to send four raw SYN packets in a controlled loopback environment, and analyzed both captures using tshark filters, counts, and expert diagnostics.

OBJECTIVES

By the end of this exercise, the learner will be able to:

- Differentiate a complete TCP three-way handshake from an incomplete or half-open connection attempt.
- Apply tshark display filters and field extraction to identify SYN, SYN-ACK, ACK, and RST packets within a capture.
- Calculate key indicators of anomalous connection behavior, including SYN counts, unique source ports, response ratios, and incomplete-handshake occurrences.
- Construct a controlled, four-packet loopback-only traffic simulation using Scapy, strictly within an authorized lab environment.
- Develop a defensible, evidence-based timeline and incident finding derived from PCAP analysis.
- Recommend appropriate detection and mitigation controls for SYN flood activity based on observed traffic patterns.

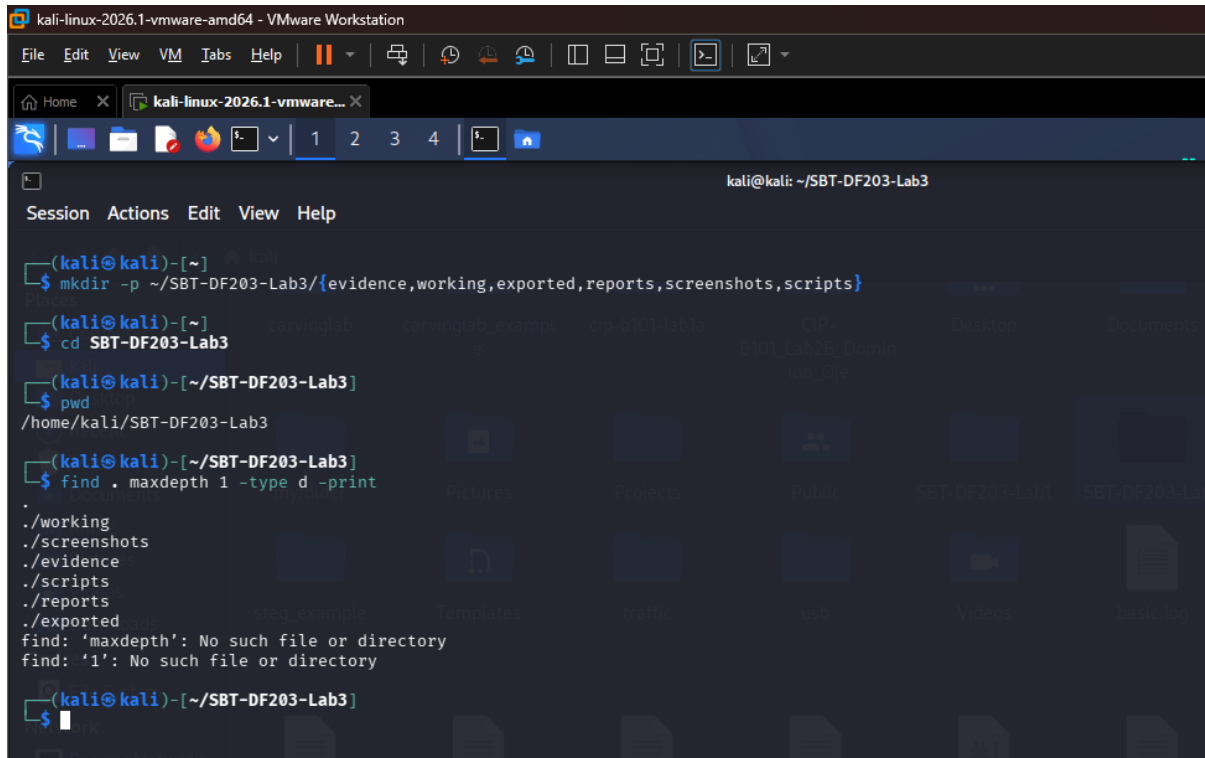
TOOLS

- Authorized training Evidence
- Kali Linux
- Microsoft Word
- Github, Git
- Md5sum and sha256sum
- Apache2
- Wireshark, tshark, Web browser, curl
- python3-scapy

METHODOLOGY

Lab Folder Structure and Evidence Preparation

The first step is to create a working environment, I created the folder structure before downloading or generating evidence of any form.

A screenshot of a Kali Linux terminal window running inside a VMware Workstation. The terminal shows the user creating a directory structure for a lab. The commands and output are as follows:

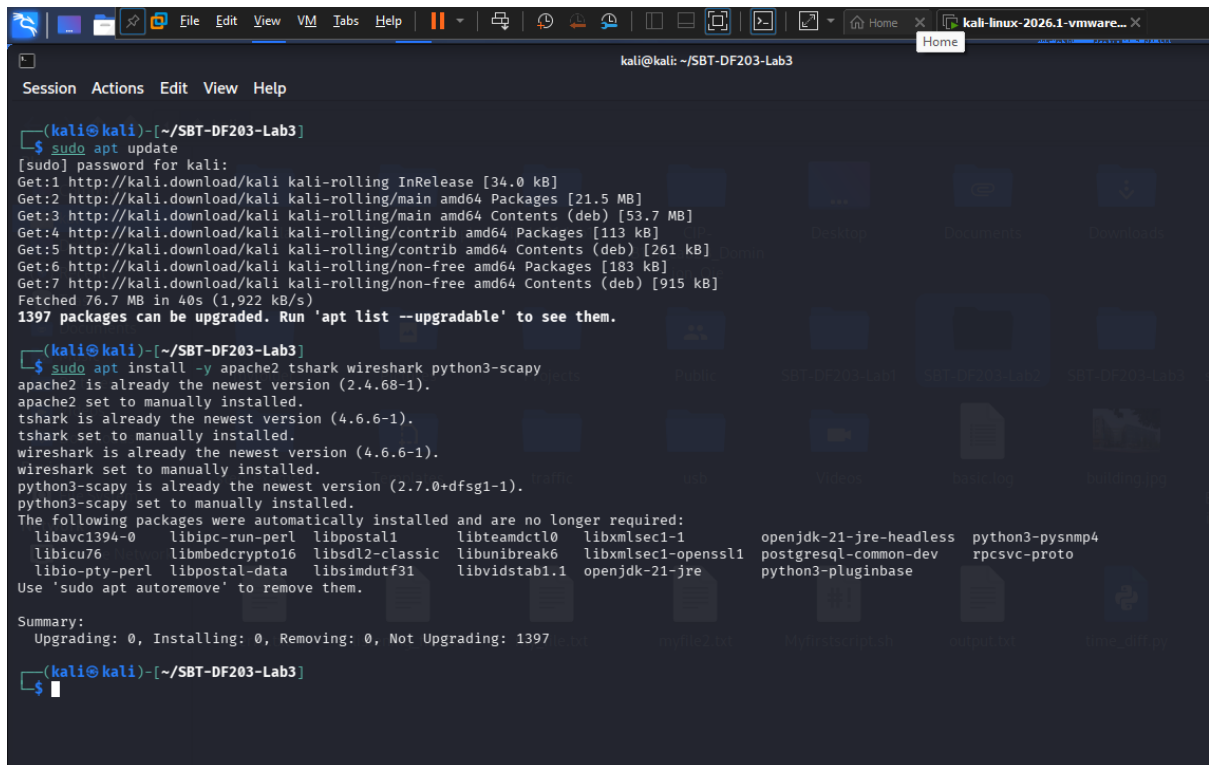
```
(kali@kali)-[~]  
$ mkdir -p ~/SBT-DF203-Lab3/{evidence,working,exported,reports,screenshots,scripts}  
(kali@kali)-[~]  
$ cd SBT-DF203-Lab3  
(kali@kali)-[~/SBT-DF203-Lab3]  
$ pwd  
/home/kali/SBT-DF203-Lab3  
(kali@kali)-[~/SBT-DF203-Lab3]  
$ find . maxdepth 1 -type d -print  
.  
./working  
./screenshots  
./evidence  
./scripts  
./reports  
./exported  
find: 'maxdepth': No such file or directory  
find: '1': No such file or directory  
(kali@kali)-[~/SBT-DF203-Lab3]  
$
```

The terminal window has a menu bar with 'Session', 'Actions', 'Edit', 'View', and 'Help'. The title bar indicates the window is titled 'kali-linux-2026.1-vmware-amd64 - VMware Workstation'. The background of the terminal shows a file manager interface with icons for various folders like 'Pictures', 'Projects', 'Public', 'SBT-DF203-Lab3', 'Templates', 'Videos', and 'Downloads'.

Fig 1: Creating the working environment

Now we then generate evidence and download the necessary tools needed to run the lab and also getting the packets that would be used for the SYN-Flood attack from the link and using the command

wget -O evidence/mySYNFloodCapture.pcap 'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/lab_files/SYN_Flood/mySYNFloodCapture.pcap'

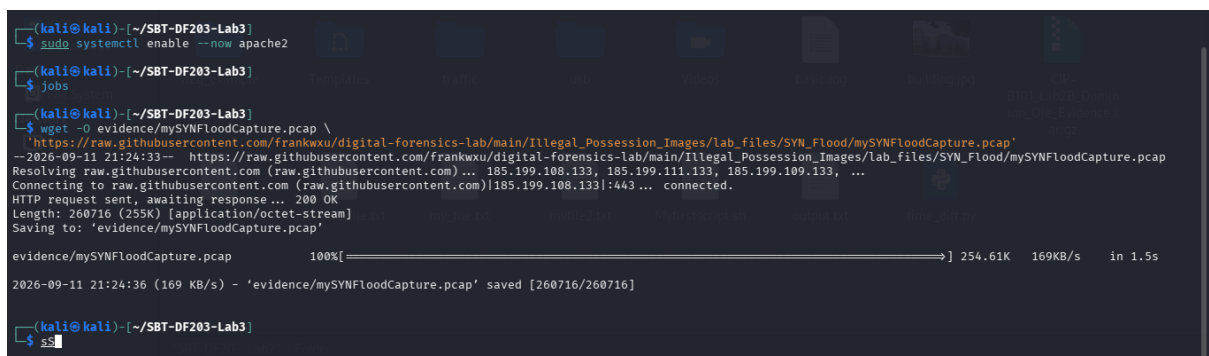
A terminal window titled 'kali@kali: ~/SBT-DF203-Lab3' showing the execution of 'sudo apt update' and 'sudo apt install -y apache2 tshark wireshark python3-scapy'. The update process shows 1397 packages can be upgraded. The installation process shows that several packages are already at the newest version and are being set to manually installed. A list of packages to be automatically removed is displayed at the bottom.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo apt update
[sudo] password for kali:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [53.7 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Fetched 76.7 MB in 40s (1,922 kB/s)
1397 packages can be upgraded. Run 'apt list --upgradable' to see them.

(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo apt install -y apache2 tshark wireshark python3-scapy
apache2 is already the newest version (2.4.68-1).
apache2 set to manually installed.
tshark is already the newest version (4.6.6-1).
tshark set to manually installed.
wireshark is already the newest version (4.6.6-1).
wireshark set to manually installed.
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-scapy set to manually installed.
The following packages were automatically installed and are no longer required:
 libavc1394-0 libipc-run-perl libpostal1 libteamdctl0 libxmlsec1-1 openjdk-21-jre-headless python3-pysnmp4
 libicu76 libmbcrypto16 libSDL2-classic libunibreak6 libxmlsec1-openssl postgresql-common-dev rpcsvc-proto
 libio-pty-perl libpostal-data libsimutf31 libvidstab1.1 openjdk-21-jre python3-pluginbase
Use 'sudo apt autoremove' to remove them.

Summary:
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1397
```

Fig 2: Updating the terminal and installing necessary tools

A terminal window showing the execution of 'sudo systemctl enable --now apache2', 'jobs', and 'wget'. The 'jobs' command shows a background process running. The 'wget' command is downloading a file from a GitHub repository. A progress bar is shown for the download.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo systemctl enable --now apache2

(kali@kali)-[~/SBT-DF203-Lab3]
$ jobs
[1] 2026-09-11 21:24:33 -- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/Lab_files/SYN_Flood/mySYNFloodCapture.pcap \
--2026-09-11 21:24:33-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Illegal_Possession_Images/Lab_files/SYN_Flood/mySYNFloodCapture.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.111.133, 185.199.109.133, ...
HTTP request sent, awaiting response... 200 OK
Length: 260716 (255K) [application/octet-stream]
Saving to: 'evidence/mySYNFloodCapture.pcap'

evidence/mySYNFloodCapture.pcap 100%[=====] 254.61K 169KB/s in 1.5s

2026-09-11 21:24:36 (169 KB/s) - 'evidence/mySYNFloodCapture.pcap' saved [260716/260716]

(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 3: Apache starts in the background, and the packets are downloaded

Then we check the Integrity by hashing the value of both the original evidence and the copy and from the image we see that the hash value is same for both the original copy and the working copy. The original copy was kept in the evidence folder and the copy for analysis was kept in the working folder.

```

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ sha256sum evidence/mySYNFloodCapture.pcap | tee reports/syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5  evidence/mySYNFloodCapture.pcap

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ ls
evidence  exported  reports  screenshots  scripts  working

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ ls -lh evidence
total 256K
-rw-rw-r-- 1 kali kali 255K Sep 11 21:24 mySYNFloodCapture.pcap

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ cp evidence/mySYNFloodCapture.pcap working/

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ ls -lh evidence
total 256K
-rw-rw-r-- 1 kali kali 255K Sep 11 21:24 mySYNFloodCapture.pcap

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ ls -lh working
total 256K
-rw-rw-r-- 1 kali kali 255K Sep 11 21:42 mySYNFloodCapture.pcap

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$ sha256sum working/mySYNFloodCapture.pcap | tee reports/Analysis_syn_capture_sha256.txt
14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5  working/mySYNFloodCapture.pcap

(kali㉿kali)-[~/SBT-DF203-Lab3]
└─$

```

Fig 4: Hashing the original and copy and checking their differences

Mini Evidence and Chain-of-Custody Worksheet

Field	Student Entry
Case/lab identifier	SBT-DF203-Lab3-Oje-Dominion-Mayowa
Trainee name	OJE DOMINION MAYOWA
Date and time started	11/09/2026, 8:00PM
Evidence file name(s)	mySYNFloodCapture.pcap,
Source or generation method	Github link,
Original SHA-256	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5
Working-copy SHA-256	14765b029a72e9c41dd8b4d32f5b1d2c7d9efee0f084949151f183a39baa55f5
Analysis workstation/VM	Kali Linux
Notes on any changes	

Part A - Establish a Normal Handshake Baseline

The first step was to start the capture on a separate terminal (terminal 1)

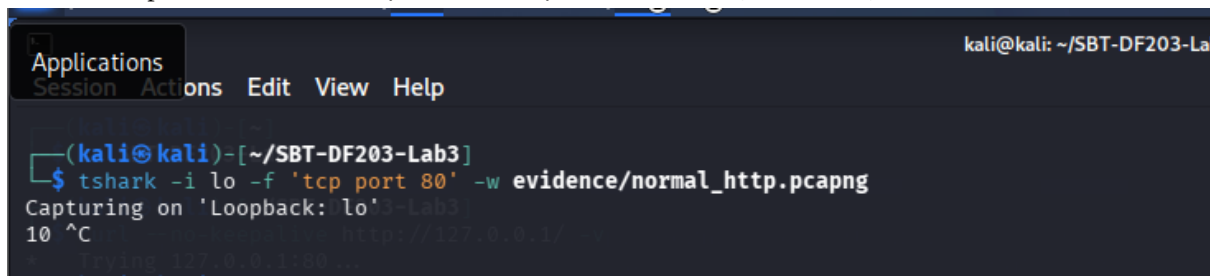


Fig 5: Start capturing or listening with tshark

Then we run the curl to run a normal handshake on another terminal (terminal 2)

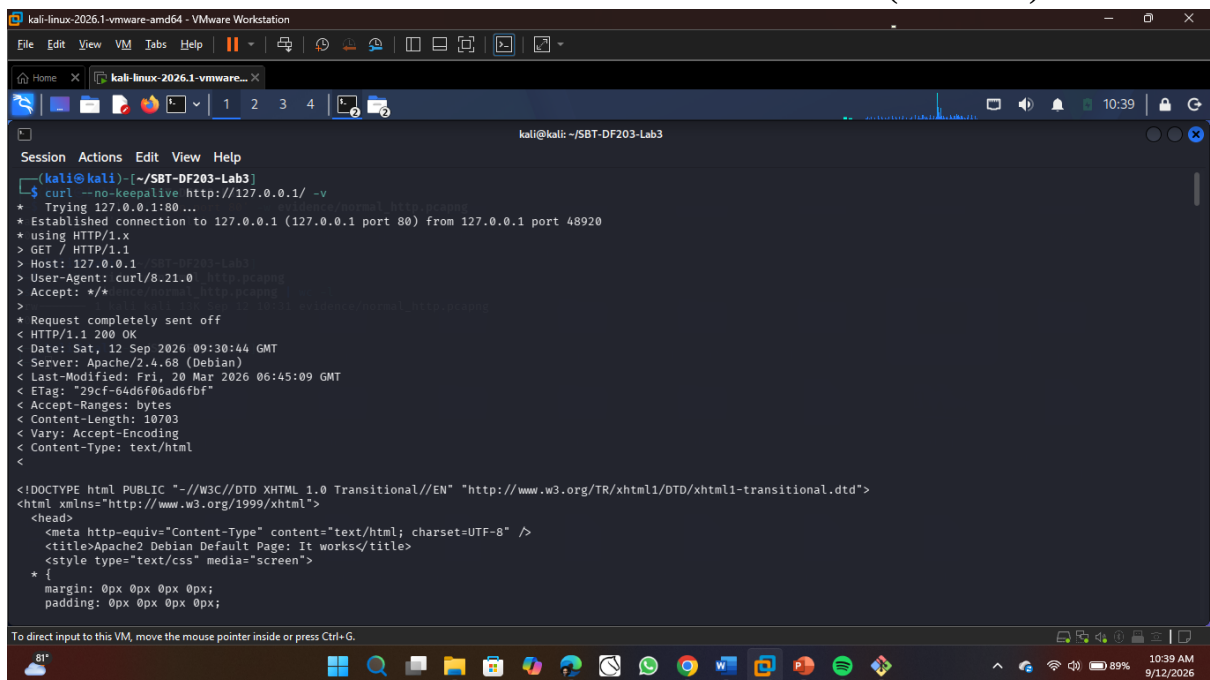


Fig 6: Sending an HTTP requests using curl

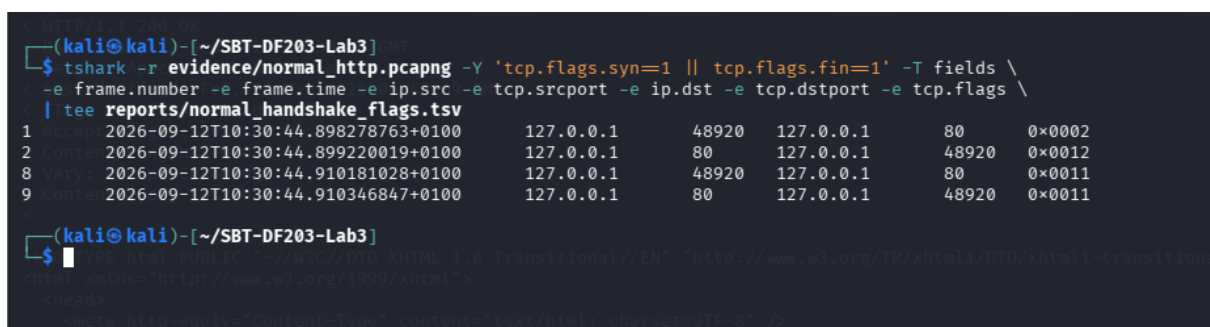


Fig 7: The handshake response as seen

A complete, healthy TCP connection follows this pattern: SYN → SYN-ACK → ACK to open the connection, followed later by FIN-ACK → FIN-ACK → ACK to close it. In this baseline capture, the handshake opened correctly (packets 1–2, with the plain ACK as packet

3 not shown here since it lacks SYN/FIN flags), and the connection was closed cleanly on both sides (packets 8–9), confirming normal, expected TCP behavior with no dropped, retransmitted, or half-open connections.

Part B - Conduct the Bounded Loopback Simulation

The first step was to create the scapy script to be able to fire 4 fake TCP connection-opening packets (SYN packets) to my local Apache server, and then stops. It doesn't wait for replies, neither does it complete any handshakes, and it doesn't keep running.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ cat > scripts/syn_probe_lab.py <<'PYLAB'
from scapy.all import IP, TCP, RandShort, send

TARGET = '127.0.0.1'
PORT = 80
COUNT = 4

packets = [IP(dst=TARGET)/TCP(sport=RandShort(), dport=PORT, flags='S') for _ in range(COUNT)]
send(packets, verbose=False)
print(f'Sent {COUNT} authorized training SYN packets to {TARGET}:{PORT}')
PYLAB
```

Fig 8: Scapy Script

Then using tshark we created a listener to capture anything going on in port 80

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -i lo -f 'tcp port 80' -c 20 -w evidence/bounded_syn_activity.pcapng
Capturing on 'Loopback: lo'
12
```

Fig 9: Started capture in terminal 1

Then the script was run and it was successful as it only sent four packets to the IP on the port

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ sudo python3 scripts/syn_probe_lab.py
[sudo] password for kali:
Sent 4 authorized training SYN packets to 127.0.0.1:80
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 10: Successfully ran the Scapy script

Part C - Identify SYN Indicators with tshark

We then identify SYN indicators with tshark and then we check the hashes of both the original variable in the evidence folder and the one in the working folder. From the image below we see that they are exactly the same.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ PCAP=working/bounded_syn_activity_working.pcapng
cp --preserve=timestamps evidence/bounded_syn_activity.pcapng "$PCAP"
sha256sum evidence/bounded_syn_activity.pcapng "$PCAP" | tee reports/bounded_capture_hashes.txt
ca9b9258300cea79ec8e4682af73c665635c9b80c016646074f778557093c575 evidence/bounded_syn_activity.pcapng
ca9b9258300cea79ec8e4682af73c665635c9b80c016646074f778557093c575 working/bounded_syn_activity_working.pcapng
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 11: Making a copy of the evidence of the SYN FLOOD

Starting with the Initial Sequences (SYN's) only with the frame number (1,4,7,10)

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq \
  | tee reports/initial_syns.tsv
1 1789207583.995134797 127.0.0.1 25203 127.0.0.1 80 0
4 1789207583.997201011 127.0.0.1 2346 127.0.0.1 80 0
7 1789207583.998584306 127.0.0.1 56628 127.0.0.1 80 0
10 1789207583.999706817 127.0.0.1 12667 127.0.0.1 80 0
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 12: Starting with the Initial SYN's being listed

Now we get the SYN-ACK responses, that is for every SYN request there is a SYN-ACK response which have the frame number (2,5,8,11)

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==1' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.ack \
  | tee reports/syn_ack_responses.tsv
2 1789207583.995898897 127.0.0.1 80 127.0.0.1 25203 1
5 1789207583.997412159 127.0.0.1 80 127.0.0.1 2346 1
8 1789207583.998604176 127.0.0.1 80 127.0.0.1 56628 1
11 1789207583.999819249 127.0.0.1 80 127.0.0.1 12667 1
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 13: SYN-ACK responses

Then we get a final ACK response which search through the capture file for packets that look like responses to the SYN probes,

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.reset==1 || (tcp.flags.ack==1 && tcp.len==0)' -T fields \
  -e frame.number -e frame.time_epoch -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags \
  | tee reports/ack_reset_candidates.tsv
2 1789207583.995898897 127.0.0.1 80 127.0.0.1 25203 0x0012
3 1789207583.995919611 127.0.0.1 25203 127.0.0.1 80 0x0004
5 1789207583.997412159 127.0.0.1 80 127.0.0.1 2346 0x0012
6 1789207583.997423311 127.0.0.1 2346 127.0.0.1 80 0x0004
8 1789207583.998604176 127.0.0.1 80 127.0.0.1 56628 0x0012
9 1789207583.998610571 127.0.0.1 56628 127.0.0.1 80 0x0004
11 1789207583.999819249 127.0.0.1 80 127.0.0.1 12667 0x0012
12 1789207583.999828231 127.0.0.1 12667 127.0.0.1 80 0x0004
(kali@kali)-[~/SBT-DF203-Lab3]
$
```

Fig 14: ACK Responses with a Reset Flag

Part D - Quantify and Correlate the Activity

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields \
  -e ip.src -e ip.dst -e tcp.dstport | sort | uniq -c | sort -nr \
  | tee reports/syn_counts_by_pair.txt
4 127.0.0.1 127.0.0.1 80
```

Fig 15: Number of Bare SYN's

This counts how many bare SYN's (no ACK) came from each source to each destination/port.


```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.flags.syn==1 && tcp.flags.ack==0' -T fields -e tcp.srcport \
  | sort -n | uniq | tee reports/unique_syn_source_ports.txt
2346
12667
25203
56628
```

Fig 16: Displays Unique Client source port

The image above lists each distinct source port used which shows 4 different random ports, one per SYN packet (since the script used RandShort()).

Then we save the reports and this pulls Wireshark's built-in "expert analysis" flags anomalies like retransmissions, malformed packets, or unusual behavior automatically.

```
(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -q -z expert | tee reports/expert_info.txt

Warns (4)
+-----+
| Frequency | Group | Protocol | Summary |
+-----+
| 4 | Sequence | TCP | Connection reset (RST) |
+-----+

Notes (4)
+-----+
| Frequency | Group | Protocol | Summary |
+-----+
| 4 | Protocol | TCP | The SYN packet does not contain a SACK PERM option |
+-----+

Chats (8)
+-----+
| Frequency | Group | Protocol | Summary |
+-----+
| 4 | Sequence | TCP | Connection establish request (SYN): server port 80 |
| 4 | Sequence | TCP | Connection establish acknowledge (SYN+ACK): server port 80 |
+-----+

(kali@kali)-[~/SBT-DF203-Lab3]
$ tshark -r "$PCAP" -Y 'tcp.analysis.retransmission || tcp.analysis.lost_segment || tcp.analysis.duplicate_ack' \
  -T fields -e frame.number -e frame.time -e _ws.col.Info | tee reports/tcp_analysis_events.tsv

(kali@kali)-[~/SBT-DF203-Lab3]
```

Fig 17: Gets a Built-in report from wireshark

Part E - Compare Normal and Suspicious Sessions

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
Initial SYN count	1	4	Multiple SYNs from the same source to the same destination in a tight time window (all within ~4 milliseconds) is a strong indicator of scripted/automated activity rather than normal user behavior
SYN-ACK count	1	4	The server responded correctly to every SYN, confirming Apache itself was healthy and behaving normally; the anomaly originates from the client side, not the server
Completed handshakes	1 (full 3-way handshake)	0	None of the 4 connections were completed with a final ACK; every one

Indicator	Normal HTTP Session	Bounded SYN Activity	Forensic Meaning
			was aborted with a RST instead, indicating half-open, abandoned connection attempts rather than legitimate sessions
Unique client source ports	1 (48920)	4 (25203, 2346, 56628, 12667)	Rapidly cycling through many distinct source ports toward the same destination:port is a classic signature of scripted connection attempts, such as scanning or flood-style activity
HTTP request present?	Yes (GET / returned full page)	No	No application-layer data was ever exchanged; the activity never advanced beyond the TCP handshake stage, confirming these were not genuine client requests
RST packets	0	4 (frames 3, 6, 9, 12)	Every connection attempt was actively reset immediately after the server's SYN-ACK; this is consistent with the source machine never intending to complete a real connection, a key fingerprint of the tooling used (Scapy raw packets bypassing the kernel's TCP stack)
Observed duration	~12ms across handshake+close	~4ms across all 4 SYN/SYN-ACK/RST cycles (timestamps 995 to 999 in the same second)	Extremely tight timing between 4 separate "connection attempts" from different ports is inconsistent with real user behavior and consistent with automated packet generation

From what I observed, the normal session went through a full handshake and an actual HTTP exchange, while the SYN probe activity showed four handshakes that never completed, each from a different source port, and each one ending in a RST packet, all happening within milliseconds of each other. This made it easy to see the clear difference between real client traffic and scripted, aborted connection attempts.